Server-side Dynamic Pages

These static web pages are so dull…

Agenda

- **Static pages vs. dynamic pages**
- Server-side dynamic pages
- Dynamic page URL routing
  - Python decorators

Static vs. dynamic content

- Static Pages
  - Show the same content every time.
  - Example: Wikipedia article pages
- Dynamic Pages
  - Content changes based on user or data
  - Example: Gmain
- Things that are impossible with simple static pages
  - Web search
  - Current time
  - # visitors to page

Project 1 = Static content

- Pages are static

- Pages only change rarely via a manual process
  - You manually update data, templates (with a text editor)
  - Run `insta485generator` to produce new pages

- Templates are a way to reduce the work of editing

- Everyone sees the same content
- Content is not generated per request
- Pages stay the same until the next manual update

Agenda

- Static pages vs. dynamic pages
- **Server-side dynamic pages**
- Dynamic page URL routing
  - Python decorators

Server-side dynamic pages

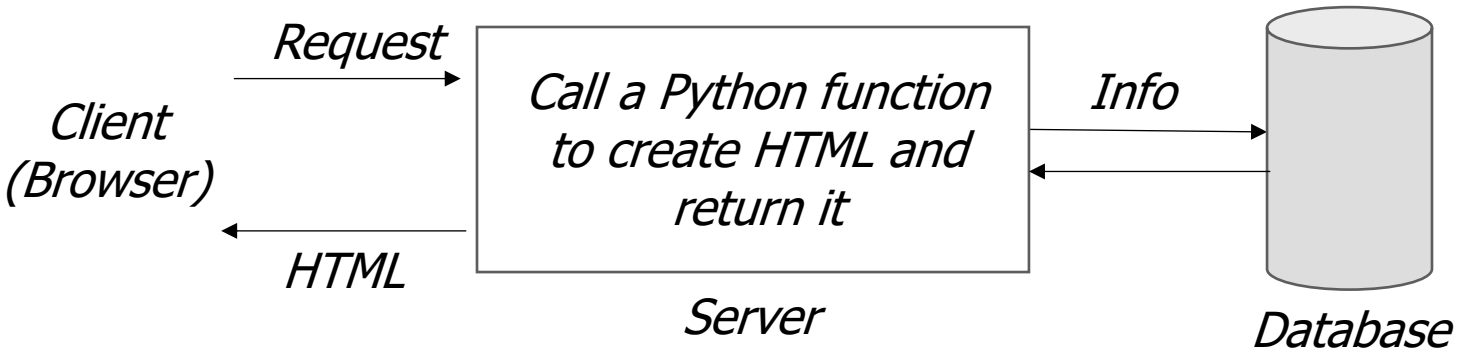
- **Key idea**: The response is the output of a function.

1. Client sends a request
2. Server runs a function
3. The function generates HTML
4. Server sends that HTML back as the response

Server-side dynamic pages

- **Key idea**: The response is the output of a function.

1. Client sends a request
2. Server runs a function
3. The function generates HTML
4. Server sends that HTML back as the response



Server-side dynamic pages example

- Flask(Python) generates web pages dynamically on the server

```
# hello.py
from flask import Flask
app =Flask(__name__)


@app.route("/hello")
def hello_world():
    return "<html><body>Hello World!</body></html>"
```

Server-side dynamic pages example

- Flask(Python) generates web pages dynamically on the server

```
# hello.py
from flask import Flask
app = Flask(__name__)


@app.route("/hello")
def hello_world():
    return "<html><body>Hello World!</body></html>"
```

Flask library for dynamic web pages

Server-side dynamic pages example

- Flask(Python) generates web pages dynamically on the server

```
# hello.py
from flask import Flask
app = Flask(__name__)
```

Flask library for dynamic web pages

Call `hello_world()` when client requests URL `/hello`

```
@app.route("/hello")
def hello_world():
    return "<html><body>Hello World!</body></html>"
```

Server-side dynamic pages example

- Flask(Python) generates web pages dynamically on the server

```
# hello.py
from flask import Flask
app =Flask(__name__)
```

Flask library for dynamic web pages

```
@app.route("/hello")
def hello_world():
    return "<html><body>Hello World!</body></html>"
```

Call `hello_world()` when client requests URL `/hello`

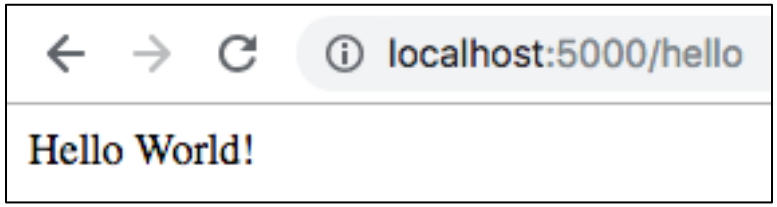
Function returns HTML as the response

Server-side dynamic pages example

- Start the server

```
$ python3 hello.py
  Running on http://127.0.0.1:5000/
```

- Browse to http://localhost:5000/hello



- localhost is the same as 127.0.0.1 (your own computer)

Templating

- Templates are a common way to generate server-side dynamic pages.
- Example: Python's `jinja2` library
- Templets let you mix:
  - HTML structure
  - Dynamic data

Template example

- Template is an HTML file containing jinja2 syntax

```
#hello.html
<html>
<head><title>Hello world</title></head>
<body>
  {% for word in words %}
    {{word}}
  {% endfor %}
</body>
</html>
```

Rendering templates with Flask

```
# hello.py
from flask import Flask
app =Flask(__name__)

@app.route("/hello")
def hello_world():
    return "<html><body>Hello World!</body></html>"
    context = {"words": ["Hello", "World!"]}
    return flask.render_template(
        "hello.html", **context)
```

Input: Template + Data
Output: Final HTML page

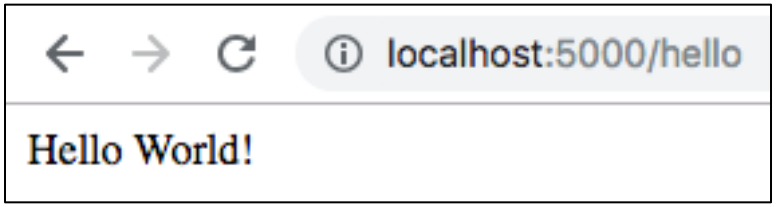
Run Flask example

- Start the server

```
$ python3 hello.py
```

- Open in browser http://localhost:5000/hello

- Flask: Runs Python function; Renders template and Returns HTML



Project 2 = Server-side dynamic content

Client requests a URL

- Looks like a file path
- But server runs a function

- How does function generate content?
  - Data is stored in a database (SQLite)
  - Queries database to get relevant data
  - Builds Python object
  - Renders template using object
  - Returns resulting HTML

- Content is generated specifically for each request

Agenda

- Static pages vs. dynamic pages
- Server-side dynamic pages
- **Dynamic page URL routing**
  - Python decorators

URL routing

- Which URLs map to which functions?
- This is called *URL routing*

- When a client requests a URL, how does a server know which function to call?

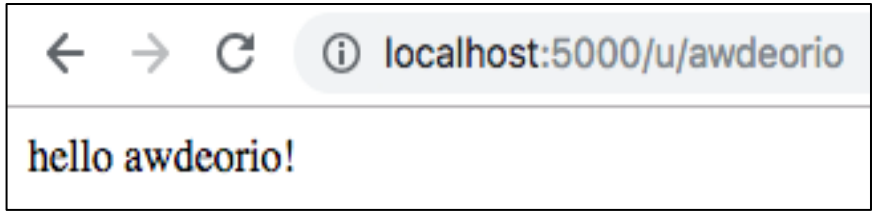
- What about routes like a user page, where the function could have an input?

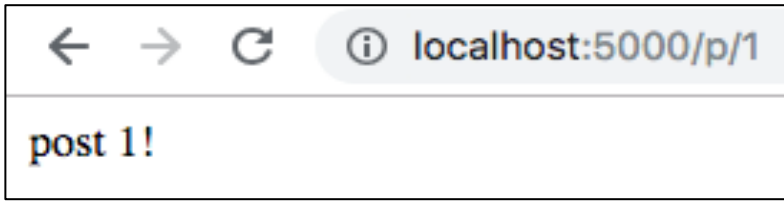
Routes with inputs

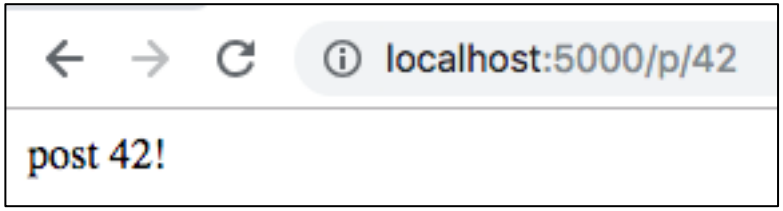
```
from flask import Flask
app = Flask(__name__)


@app.route('/u/<username>')
def show_user(username):
    return "hello " + username + "!"


@app.route('/p/<postid>')
def show_post(postid):
    return "post "+ postid  + "!"
```







Static routes vs. dynamic routes

- Static pages: one URL maps to one file
  - Example: /index.html

- Dynamic pages: many URLs map to one function
  - Example: https://localhost:5000/u/John calls `show_user("John")`
  - Example: https://localhost:5000/u/smith calls `show_user("smith")`

```
@app.route('/u/<username>')
def show_user(username):
    return "hello " + username + "! "
```

URL routing

- Which URLs map to which functions?
- This is called *URL routing*

- **Routing table** mapping URL to function-reference
  - /u/ show_user
  - /p/ show_post

- When a request comes in, Flask:
  - Finds the matching pattern
  - Gets the function
  - Calls the function

Routing (Decorators)

- Flask uses a Python ***decorator*** to describe routing

- A decorator is a higher order function

```
@app.route('/u/<username>')
def show_user(username):
    return "hello “ + username +”!"
```

Why this works: First Class Functions

- In Python, functions are *first class objects*
- Recall from EECS 280, that first class objects can be:
  - Passed as input
  - Returned as output
  - Created at runtime
  - Destroyed at runtime

Create Function at runtime

- Function object contains function execution "code“ created when function is declared

```
def show_user(username):
    return "hello "+username+ "!"

output = show_user("John")
print(output)
```

Pass a function as input

```
def show_user(username):
    return "hello”+ username+ ”!"

def show_post(postid):
    return "post”+ postid+ ”!"

def show(func, var):
    print(func(var))

show(show_user, “Smith")
show(show_post, 1)
```

Pass a function as input

```
def show_user(username):
    return "hello " + username+ "!"

def show_post(postid):
    return "post "+ postid+ "!"

def show(func, var):
    print(func(var))

show(show_user, "Smith")
show(show_post, 1)
```

```
$ python3 test.py
hello Smith!
post 1!
```

Return a function as output

```
def get_view(url):
    if url.startswith("/u"):
        return show_user
    elif url.startswith("/p"):
        return show_post
    else:
        return None

def view(baseurl, slug):
    func = get_view(baseurl)
    print(func(slug))

view("/u", "Smith")
```

```
def show_user(username):
    return "hello ..."

def show_post(postid):
    return "post ..."
```

```
$ python3 test.py
hello Smith!
```

Return a function as output

```
def get_view(url):
    if url.startswith("/u"):
        return show_user
    elif url.startswith("/p"):
        return show_post
    else:
        return None
```

```
def show_user(username):
    return "hello ..."

def show_post(postid):
    return "post ..."
```

/u */Smith*

```
def view(baseurl, slug):
    func = get_view(baseurl)
    print(func(slug))

view("/u", "Smith")
```

```
$ python3 test.py
hello Smith!
```

Return a function as output

```
                /u
def get_view(~~url~~):
    if url.startswith("/u"):
        return show_user
    elif url.startswith("/p"):
        return show_post
    else:
        return None

              /u      /Smith
def view(~~baseurl~~, ~~slug~~):
    func = get_view(baseurl)
    print(func(slug))

view("/u", "Smith")
```

```
def show_user(username):
    return "hello ..."

def show_post(postid):
    return "post ..."
```

```
$ python3 test.py
hello Smith!
```

Return a function as output

```
                /u
def get_view(~~url~~):
    if url.startswith("/u"):
        return show_user
    elif url.startswith("/p"):
        return show_post
    else:
        return None

              /u        /Smith
def view(~~baseurl~~, ~~slug~~):
    func = get_view(baseurl)
    print(func(slug))

view("/u", "Smith")
```

```
def show_user(username):
    return "hello ..."

def show_post(postid):
    return "post ..."
```

```
$ python3 test.py
hello Smith!
```

Return a function as output

```
                /u
def get_view(~~url~~):
    if url.startswith("/u"):
        return show_user
    elif url.startswith("/p"):
        return show_post
    else:
        return None

              /u      /Smith
def view(~~baseurl~~, ~~slug~~):
    func = ~~get_view(baseurl)~~ Show_user
    print(func(slug))

view("/u", "Smith")
```

```
def show_user(username):
    return "hello ..."

def show_post(postid):
    return "post ..."
```

```
$ python3 test.py
hello Smith!
```

Connecting to routing

- Flask routing is conceptually:

- **Routing table** mapping URL to function-reference
  - /u/ show_user
  - /p/ show_post

```
func = routing_table[url]
func(parameter)
```

- Routing = choose a function based on the URL, then call it.

Agenda

- Static pages vs. dynamic pages
- Server-side dynamic pages
- Dynamic page URL routing
  - **Python decorators**

Decorator

- Flask routing uses decorators to connect URLs to functions.

- A decorator is a function that **takes a function and returns a function**

- It **wraps** another function to add or modify behavior

```
@app.route('/u/<username>')
def show_user(username):
    return "hello " + username + "! "
```

Simple decorators

```
def register(func):
    print("registered ” + func.__name__)
    return func

def show_user(username):
    return "hello " + username + "!"

show_user = register(show_user)

print(show_user(“smith"))
```

What is the output?

Simple decorators

```
def register(func):
    print("register ” + func.__name__)
    return func

def show_user(username):
    return "hello “ + username +”!"

# "Manual" decorator happens here
show_user = register(show_user)

print(show_user(“smith"))
```

```
$ python3 test.py
Register show_user
hello smith!
```

Decorator syntactic sugar

```
def register(func):
    print("register” + func.__name__)
    return func

@register # Python decorator syntax
def show_user(username):
    return "hello “ + username +”!"
```

~~show_user = register(show_user)~~

```
print(show_user(“smith"))
```

```
$ python3 test.py
Register show_user
hello smith!
```

Summary

- **Static pages**: Server response is the same every time
- **Server-side dynamic pages**: Server response is the output of a function
- **URL routing:** When a client requests a server-side dynamic pages URL, server *routes* the request to a function
- **Decorator** is a function that transforms another function and used by Python/Flask for URL routing